

Building a GET API : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-v2-building-get-api>  <https://github.com/learn-co-curriculum/python-p4-v2-building-get-api/issues/new>

Learning Goals

- Build an API to handle GET requests.
- Serialize objects
- Implement an association proxy

Key Vocab

- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

Imagine this scenario: you're given the task of creating a new game review website from scratch. You want a dynamic, highly interactive frontend, so naturally you choose React. You also need to store the data about your users, your games, and the reviews somewhere. Well, it sounds like we need a database for that. Great! We can use SQLAlchemy to set up and access data from the database.

Here's the problem though. React can't communicate directly with the database — for that, you need SQLAlchemy and Flask. SQLAlchemy also doesn't know anything about your React application (and nor should it!). So then how can we connect up our React frontend with the database?

Well, it sounds like we need some sort of **interface** between React and our database. Perhaps some sort of **Application Programming Interface** (or as you may know it, API). We need a structured way for these two applications to communicate, using a couple things they **do** have in common: **HTTP** and **JSON**.

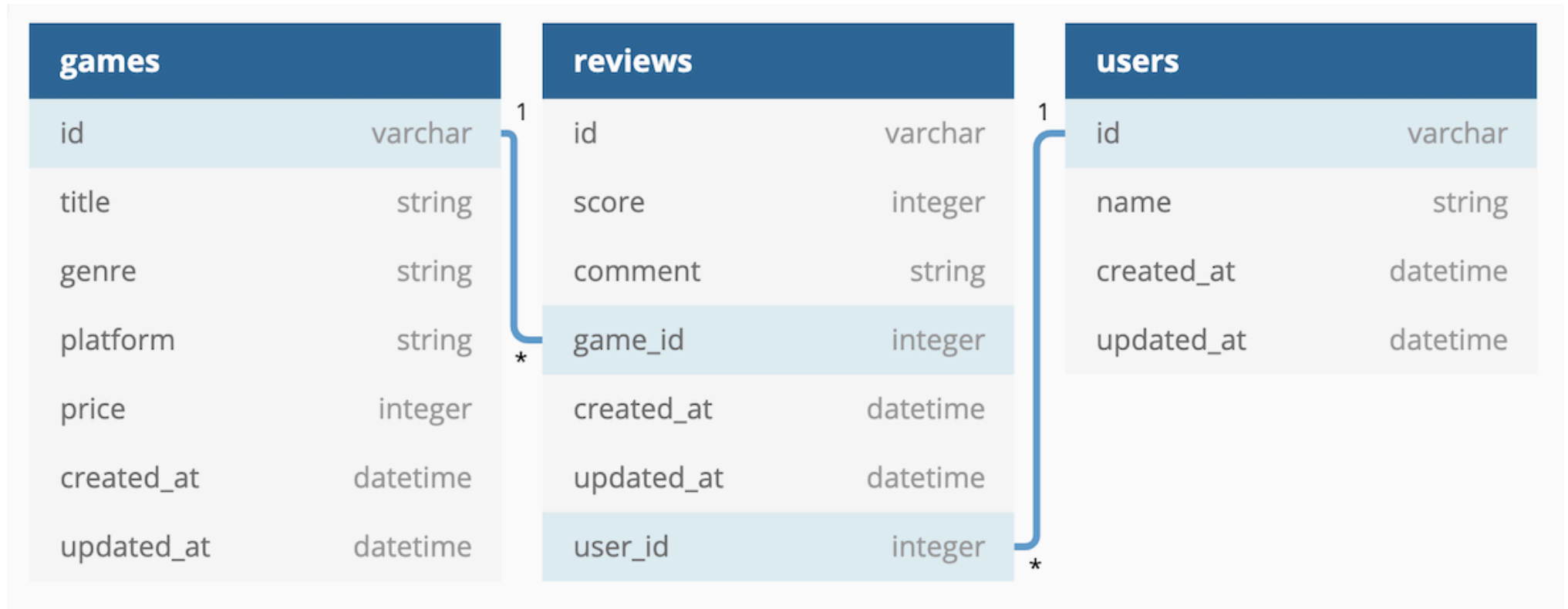
That is what we'll be building for the rest of this section: an API (specifically, a JSON API) that will allow us to use SQLAlchemy to communicate with a database from a React application — or really, from any application that speaks HTTP!

Setup

We'll build up our Flask application from a few models and views that are ready to go. Run these commands to install the dependencies and set up the database:

```
$ pipenv install; pipenv shell
$ cd server
$ flask db init
$ flask db migrate -m 'initial migration'
$ flask db upgrade head
$ python seed.py
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

You can view the models in the `server/models.py` module, and the migrations in the `server/migrations/versions` directory. Here's what the relationships will look like in our ERD:



Then, run the server:

```
$ python app.py
```

With that set up, let's work on getting Flask and SQLAlchemy working together!

Getting All Records

Imagine we're building a feature in a React application where we'd like to show our users a list of all the games in the database. From React, we might have code similar to the following to make this request for the data:

```
function GameList() {
  const [games, setGames] = useState([]);

  useEffect(() => {
    fetch("http://localhost:9292/games")
      .then((r) => r.json())
      .then((games) => setGames(games));
  }, []);

  return (
    <section>
      {games.map((game) => (
        <GameItem key={game.id} game={game} />
      ))}
    </section>
  );
}
```

It's now our job to set up the server so that when a GET request is made to `/games` , we return an array of all the games in our database in JSON format. Let's set that up in Flask by adding a view named `games()` :

```
# server/app.py
```

```
from flask import Flask, jsonify, make_response
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

from models import db, User, Review, Game

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///app.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
```

```
app.json.compact = False

migrate = Migrate(app, db)

db.init_app(app)

@app.route('/')
def index():
    return "Index for Game/Review/User API"

@app.route('/games')
def games():

    games = []
    for game in Game.query.all():
        game_dict = {
            "title": game.title,
            "genre": game.genre,
            "platform": game.platform,
            "price": game.price,
        }
        games.append(game_dict)

    response = make_response(
        jsonify(games),
        200
    )

    return response

if __name__ == '__main__':
```

```
app.run(port=5555, debug=True)
```

We see a lot of familiar faces here, but there are a couple new elements to explore:

- `jsonify` is a method in Flask that *serializes* its arguments as JSON and returns a `Response` object. It can accept lists or dictionaries as arguments. Unfortunately, it will not accept models as arguments (darn!).
- `app.json.compact = False` is a configuration that has JSON responses print on separate lines with indentation. This adds some overhead, but if human eyes will be looking at your API, it's always good to have this set to `False`.
- Our query results have to be reformatted as dictionaries for `jsonify` to work its magic. The `__dict__` attribute cannot be used here because SQLAlchemy records have attributes that are nonstandard Python objects. We're leaving `game.id` out here because `game.title` is already set to unique.

NOTE: `jsonify()` is now run automatically on all dictionaries returned by Flask views. We'll just pass in those dictionaries from now on, but remember what `jsonify()` 's doing for you behind the scenes!

Rerun `python app.py` in the console from the `server/` directory, then navigate to <http://127.0.0.1:5555/games>  (<http://127.0.0.1:5555/games>) and you should see something similar to the following:

```
[
  {
    "genre": "Survival",
    "platform": "XBox",
    "price": 30,
    "title": "Mega Adventure"
  },
  {
    "genre": "Sports",
    "platform": "PlayStation",
    "price": 20,
    "title": "Golf Pro IV"
  },
]
```

```
{
  "genre": "Party",
  "platform": "PlayStation",
  "price": 7,
  "title": "Dance, dance, dance"
}
```

You also have a lot of control over how this data is returned by using SQLAlchemy. For example, you could sort the games by title instead of the default sort order:

```
# example
games_by_title = Game.query.order_by(Game.title).all()
```

Or just return the first 2 games:

```
# example
first_2_games = Game.query.limit(2).all()
```

Now that you have full control over how the server handles the response, you have the freedom to design your API as you see fit — just think about what kind of data you need for your frontend application.

Let's make another small adjustment to our view. By default, Flask sets a [response header](https://developer.mozilla.org/en-US/docs/Glossary/Response_header) `Content-Type: text/html`, since in general, web servers are used to send HTML content to browsers. Our server, however, will be used to send JSON data, as you've seen above. We can indicate this by changing the response header for all our routes by adding this to the `games()` view:

```
# server/app.py

# import, config, index

@app.route('/games')
```

```
def games():  
  
    games = []  
    for game in Game.query.all():  
        game_dict = {  
            "title": game.title,  
            "genre": game.genre,  
            "platform": game.platform,  
            "price": game.price,  
        }  
        games.append(game_dict)  
  
    response = make_response(  
        games,  
        200,  
        {"Content-Type": "application/json"}  
    )  
  
    return response
```

...though this is unnecessary in this case due to `jsonify()` ! So we can leave `jsonify()` and the `Content-Type` out when passing a dictionary to `make_response` .

Getting One Game Using Params

We've got our API set up to handle one feature so far: we can return a list of all the games in the application. Let's imagine we're building another frontend feature; this time, we want a component that will just display the details about one specific game, including its associated reviews. Here's how that component might look:

```
function GameDetail({ gameId }) {  
    const [game, setGame] = useState(null);
```



```

useEffect(() => {
  fetch(`http://localhost:9292/games/${gameId}`)
    .then((r) => r.json())
    .then((game) => setGame(game));
}, [gameId]);

if (!game) return <h2>Loading game data...</h2>;

return (
  <div>
    <h2>{game.title}</h2>
    <p>Genre: {game.genre}</p>
    <h4>Reviews</h4>
    {game.reviews.map((review) => (
      <div>
        <h5>{review.user.name}</h5>
        <p>Score: {review.score}</p>
        <p>Comment: {review.comment}</p>
      </div>
    ))}
  </div>
);
}

```

So for this feature, we know our server needs to be able to handle a GET request to return data about a specific game, using the game's ID to find it in the database. For example, a `GET /games/1` request should return the game with the ID of 1 from the database.

As we saw in the previous module, we can access data from the dynamic portion of the URL by using the dynamic parameter name as an argument for the view. After this, we will access the unique record using SQLAlchemy's `query.filter()` method and returning the first result.

NOTE: to retrieve the one, correct result for your filter statement, you must use a *unique* attribute. Make sure you only use unique attributes in your dynamic URLs!

Update `app.py` to add an additional view to get the game by id:

```
# server/app.py

# import, config, index, games

@app.route('/games/<int:id>')
def game_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    game_dict = {
        "title": game.title,
        "genre": game.genre,
        "platform": game.platform,
        "price": game.price,
    }

    response = make_response(
        game_dict,
        200
    )

    return response
```

With this code in place in the controller, try accessing the data about one game in the browser at <http://127.0.0.1:5555/games/1>  (<http://127.0.0.1:5555/games/1>). You should see an object like this in the response:

```
{
  "genre": "Survival",
  "platform": "XBox",
  "price": 30,
```

```
"title": "Mega Adventure"  
}
```

Try making requests using other game IDs as well. As long as the ID exists in the database, you'll get a response.

Accessing Associated Data

Right now, our server is returning information about the game, but how can we also access data about its associated models like the users and reviews? We could make another endpoint for the user and review data, and make additional requests from the frontend, but that might get messy. It would be more efficient to return this data together along with the game data in just one single response.

Let's take a look at the JSON being returned from the server. How does this Python code:

```
game = Game.query.filter(Game.id == id).first()
```

...turn into this JSON object?

```
{  
  "genre": "Racing",  
  "platform": "NES",  
  "price": 29,  
  "title": "Beat go these."  
}
```

When we're using the `jsonify()` method, Flask serializes (converts from one format to another) the SQLAlchemy object into a JSON object by getting a list of keys and values to pass to the client.

There are some clear limitations to this strategy. First, we need to write new code for every attribute that we want to share with the client. This isn't very scalable, since every new attribute requires a new line of code in `models.py` and `app.py`. Additionally, `jsonify` is very fussy with even slightly complex Python objects. (You may have already noticed this if you tried to include a `DateTime` field earlier.)

Lucky for us, there is a better way. SQLAlchemy-serializer is configured inside of our models, after which the model's attributes based on the column names are easily mapped to a dictionary that we can use to create a JSON response. We can define exactly what we want to serve to the client and how we want to serve it.

Let's modify `models.py` to serialize the `Game` model:

```
# server/models.py

from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import MetaData
from sqlalchemy_serializer import SerializerMixin
from sqlalchemy.ext.associationproxy import association_proxy

metadata = MetaData(
    naming_convention={
        "fk": "fk_(table_name)s_(column_0_name)s_(referred_table_name)s",
    }
)

db = SQLAlchemy(metadata=metadata)

class Game(db.Model, SerializerMixin):
    __tablename__ = "games"

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String, unique=True)
    genre = db.Column(db.String)
    platform = db.Column(db.String)
    price = db.Column(db.Integer)
    created_at = db.Column(db.DateTime, server_default=db.func.now())
    updated_at = db.Column(db.DateTime, onupdate=db.func.now())
```

```
reviews = db.relationship("Review", back_populates="game")

def __repr__(self):
    return f"<Game {self.title} for {self.platform}>"

# review, user
```

The simple inclusion of `SerializerMixin` adds a `to_dict()` instance method to the `Game` model. Run `python debug.py` to open up an interactive shell and enter the following:

```
ipdb> from models import *
ipdb> game = Game.query.first()
ipdb> game.to_dict()
Can not serialize type:Review
Can not serialize type:Review
{'id': 1, 'platform': 'XBox', 'title': 'Mega Adventure', 'price': 30, 'reviews': [], 'genre': 'Survival', 'created_c
ipdb>
```

Just like that, we have a dictionary representation of our first game! There's still a problem, though: the game's reviews can't be serialized!

In order to serialize models with related models, all models in the relationship tree must be configured with `SerializerMixin`. Additionally, we need to set rules to make sure that we aren't accidentally *recursively* serializing: we don't want a game's reviews' game's reviews, after all.

Troubleshooting Tips

- If you end up in a **recursive loop**, you'll see an error like the one below. You will need to tweak your serialization rules to avoid nested undesired data. For example, if you want to include a game's reviews in the serialized data but the review serializer is also trying to include the game, you'll end up in a recursive loop.

RecursionError: maximum recursion depth exceeded while calling a Python object

- The same principle should be applied when dealing with unusual responses while testing our API with Postman. If despite crossing all the t's and dotting all the i's, you still get a simple string letter rather than your JSON data, it's likely that you're in a recursive loop. Check your serialization rules and make sure that you're not trying to serialize a model that is already being serialized.

Let's modify all of our models to set them up for serialization by adding `SerializerMixin` as a superclass:

```
# imports, metadata, db

class Game(db.Model, SerializerMixin):
    __tablename__ = "games"

    serialize_rules = ("-reviews.game",)

    # columns, repr

class Review(db.Model, SerializerMixin):
    __tablename__ = "reviews"

    serialize_rules = ("-game.reviews", "-user.reviews",)

    # columns, repr

class User(db.Model, SerializerMixin):
    __tablename__ = "users"

    serialize_rules = ("-reviews.user",)

    # columns, repr
```

Now rerun `debug.py` and enter the same command. You should see all of the first game's reviews!

```
ipdb> from models import *
ipdb> game = Game.query.first()
```

```
ipdb> game.to_dict()
{'created_at': '2023-08-15 20:05:44', 'title': 'Mega Adventure', 'updated_at': None, 'genre': 'Survival', 'id': 1, 'platform': 'XBox', 'price': 30, 'reviews': []}
```

Let's reconfigure the `games/<int:id>` view to show reviews with our new, simpler strategy for serialization:

```
# server/app.py
@app.route('/games/<int:id>')
def game_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    game_dict = game.to_dict()

    response = make_response(
        game_dict,
        200
    )

    return response
```

Run your server again with `flask run` and navigate to <http://127.0.0.1:5555/games/1>  (<http://127.0.0.1:5555/games/1>) to see your fully fleshed-out `GET` API (user names are random):

```
{
  "created_at": "2023-08-15 20:05:44",
  "genre": "Survival",
  "id": 1,
  "platform": "XBox",
  "price": 30,
  "reviews": [
    {
      "comment": "Amazing action",
      "created_at": "2023-08-15 20:05:44",
```

```
    "game_id": 1,
    "id": 1,
    "score": 9,
    "updated_at": null,
    "user": {
      "created_at": "2023-08-15 20:05:44",
      "id": 1,
      "name": "Jose Coleman",
      "updated_at": null
    },
    "user_id": 1
  },
  {
    "comment": "Not enough levels",
    "created_at": "2023-08-15 20:05:44",
    "game_id": 1,
    "id": 3,
    "score": 5,
    "updated_at": null,
    "user": {
      "created_at": "2023-08-15 20:05:44",
      "id": 2,
      "name": "Joshua Brown",
      "updated_at": null
    },
    "user_id": 2
  }
],
"title": "Mega Adventure",
"updated_at": null
}
```

Let's also update the `games` view that lists all games to take advantage of the `to_dict()` method:


```
@app.route('/games')
def games():

    games = []
    for game in Game.query.all():
        game_dict = game.to_dict()
        games.append(game_dict)

    response = make_response(
        games,
        200,
        {"Content-Type": "application/json"}
    )

    return response
```

You should still see all games listed with their reviews using the route <http://127.0.0.1:5555/games>  (<http://127.0.0.1:5555/games>).

We can also take advantage of **list comprehensions** to make the code succinct:

```
@app.route('/games')
def games():

    games = [game.to_dict() for game in Game.query.all()]

    response = make_response(
        games,
        200
    )

    return response
```

Association Proxy - Users Who've Reviewed a Game

Suppose we'd like to get users who have reviewed a particular game. Currently, the user information is deeply nested within the JSON string returned when we get a game for an id (names are random):

<http://127.0.0.1:5555/games/1> ➔ <http://127.0.0.1:5555/games/1>

```
{
  "created_at": "2023-08-15 20:05:44",
  "genre": "Survival",
  "id": 1,
  "platform": "XBox",
  "price": 30,
  "reviews": [
    {
      "comment": "Amazing action",
      "created_at": "2023-08-15 20:05:44",
      "game_id": 1,
      "id": 1,
      "score": 9,
      "updated_at": null,
      "user": {
        "created_at": "2023-08-15 20:05:44",
        "id": 1,
        "name": "Jose Coleman",
        "updated_at": null
      },
      "user_id": 1
    },
    {
      "comment": "Not enough levels",
      "created_at": "2023-08-15 20:05:44",
```

```

    "game_id": 1,
    "id": 3,
    "score": 5,
    "updated_at": null,
    "user": {
      "created_at": "2023-08-15 20:05:44",
      "id": 2,
      "name": "Joshua Brown",
      "updated_at": null
    },
    "user_id": 2
  }
],
"title": "Mega Adventure",
"updated_at": null
}

```

Let's add a view that returns just the user data for a given game. Without an association proxy, we'll have to iterate through the list of reviews to get each user. Note that we are excluding the `reviews` from each user by passing a rule into the `to_dict()` method:

```

@app.route('/games/users/<int:id>')
def game_users_by_id(id):
    game = Game.query.filter(Game.id == id).first()
    users = []
    for review in game.reviews:
        user = review.user
        user_dict = user.to_dict(rules=("-reviews",))
        users.append(user_dict)

    response = make_response(
        users,
        200
    )

```

`return` response

Let's test this with <http://127.0.0.1:5555/games/users/1>  <http://127.0.0.1:5555/games/users/1>. Your names will be different.

```
[
  {
    "created_at": "2023-08-15 20:05:44",
    "id": 1,
    "name": "Jose Coleman",
    "updated_at": null
  },
  {
    "created_at": "2023-08-15 20:05:44",
    "id": 2,
    "name": "Joshua Brown",
    "updated_at": null
  }
]
```

We can simplify the code a bit using a list comprehension, but notice we still have to go through the review to get the user:

```
@app.route('/games/users/<int:id>')
def game_users_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    users = [review.user.to_dict(rules=('-reviews',))]
              for review in game.reviews]
    response = make_response(
        users,
        200
    )
```

```
return response
```

Let's update the `Game` model to add an association proxy to get the user from each review:

```
class Game(db.Model, SerializerMixin):
    __tablename__ = "games"

    serialize_rules = ("-reviews.game",)

    # columns and relationship ...

    # Association proxy to get users for this game through reviews
    users = association_proxy("reviews", "user",
                             creator=lambda user_obj: Review(user=user_obj))
```

Now we can update the view to access users for a game by `game.users` :

```
@app.route('/games/users/<int:id>')
def game_users_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    # use association proxy to get users for a game
    users = [user.to_dict(rules=("-reviews",)) for user in game.users]
    response = make_response(
        users,
        200
    )

    return response
```

Conclusion

In this lesson, you created your very first web API! You learned how to set up multiple routes to handle different requests based on what kind of data we needed for a frontend application, and used `jsonify()` and `SQLAlchemy-serializer` to serialize the JSON response to include all the data needed. You also wrote an association proxy to reach across an intermediary object in a many-to-many relationship.

At their most basic levels, almost all web APIs provide a way for clients, like React applications, to interact with a database and gain access to data in a structured way. Thanks to tools like Flask and SQLAlchemy, setting up this interface is fairly straightforward.

Solution Code

```
# server/app.py

from flask import Flask, jsonify, make_response
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

from models import db, User, Review, Game

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///app.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
app.json.compact = False

migrate = Migrate(app, db)

db.init_app(app)

@app.route('/')
def index():
    return "Index for Game/Review/User API"
```

```
# start building your API here
@app.route('/games')
def games():

    games = [game.to_dict() for game in Game.query.all()]

    response = make_response(
        games,
        200
    )

    return response

@app.route('/games/<int:id>')
def game_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    game_dict = game.to_dict()

    response = make_response(
        game_dict,
        200
    )

    return response

@app.route('/games/users/<int:id>')
def game_users_by_id(id):
    game = Game.query.filter(Game.id == id).first()

    # use association proxy to get users for a game
    users = [user.to_dict(rules=("-reviews",)) for user in game.users]
```

```
        response = make_response(
            users,
            200
        )

    return response

if __name__ == '__main__':
    app.run(port=5555, debug=True)

# server/models.py

from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import MetaData
from sqlalchemy_serializer import SerializerMixin
from sqlalchemy.ext.associationproxy import association_proxy

metadata = MetaData(
    naming_convention={
        "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
    }
)

db = SQLAlchemy(metadata=metadata)

class Game(db.Model, SerializerMixin):
    __tablename__ = "games"

    serialize_rules = ("-reviews.game",)

    id = db.Column(db.Integer, primary_key=True)
```



```
title = db.Column(db.String, unique=True)
genre = db.Column(db.String)
platform = db.Column(db.String)
price = db.Column(db.Integer)
created_at = db.Column(db.DateTime, server_default=db.func.now())
updated_at = db.Column(db.DateTime, onupdate=db.func.now())

reviews = db.relationship("Review", back_populates="game")

# Association proxy to get users for this game through reviews
users = association_proxy("reviews", "user",
                          creator=lambda user_obj: Review(user=user_obj))

def __repr__(self):
    return f"<Game {self.title} for {self.platform}>"

class Review(db.Model, SerializerMixin):
    __tablename__ = "reviews"

    serialize_rules = ("-game.reviews", "-user.reviews",)

    id = db.Column(db.Integer, primary_key=True)
    score = db.Column(db.Integer)
    comment = db.Column(db.String)
    created_at = db.Column(db.DateTime, server_default=db.func.now())
    updated_at = db.Column(db.DateTime, onupdate=db.func.now())

    game_id = db.Column(db.Integer, db.ForeignKey("games.id"))
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"))

    game = db.relationship("Game", back_populates="reviews")
    user = db.relationship("User", back_populates="reviews")
```

```
def __repr__(self):  
    return f"<Review ({self.id}) of {self.game}: {self.score}/10>"
```

```
class User(db.Model, SerializerMixin):  
    __tablename__ = "users"  
  
    serialize_rules = ("-reviews.user",)  
  
    id = db.Column(db.Integer, primary_key=True)  
    name = db.Column(db.String)  
  
    created_at = db.Column(db.DateTime, server_default=db.func.now())  
    updated_at = db.Column(db.DateTime, onupdate=db.func.now())  
  
    reviews = db.relationship("Review", back_populates="user")  
  
def __repr__(self):  
    return f"<User ({self.id}) {self.name}>"
```

Resources

- [Flask - Pallets](https://flask.palletsprojects.com/en/2.2.x/) ➞ (https://flask.palletsprojects.com/en/2.2.x/)
- [GET - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)
- [flask.json.jsonify Example Code - Full Stack Python](https://www.fullstackpython.com/flask-json-jsonify-examples.html) ➞ (https://www.fullstackpython.com/flask-json-jsonify-examples.html)
- [SQLAlchemy-serializer - PyPI](https://pypi.org/project/SQLAlchemy-serializer/) ➞ (https://pypi.org/project/SQLAlchemy-serializer/)

This tool needs to be loaded in a new browser window

Load Building a GET API : Code-Along (CodeGrade) in a new window